

# Neural Algorithmic Reasoning for solving the uncapacitated facility location problem

Šimon Rokůsek<sup>1</sup>

<sup>1</sup>University of Trento

## Abstract

The uncapacitated facility location problem requires selecting facilities and assigning clients while minimizing facility-opening and connection costs. Although exact optimization provides optimal solutions, its computational cost can become prohibitive as the number of clients and candidate facilities increases. This work investigates whether Neural Algorithmic Reasoning can learn a scalable approximation procedure for this problem by imitating the Jain-Vazirani primal-dual algorithm. A custom graph-based encoder-processor-decoder architecture is trained using intermediate algorithm traces, teacher forcing, and final solution-cost supervision obtained from an exact solver. Despite being trained only on small metric instances, the model maintains comparatively stable solution quality on configurations up to (500, 500) and exhibits scaling behavior similar to its teacher on instances up to (1000, 1000). For sufficiently large instances, NAR inference is also substantially faster than both exact optimization and the implemented teacher algorithm. Under a demand-weighted distribution shift, the model retains its solution quality and outperforms its teacher. However, performance deteriorates when the connection costs are replaced by fully random values, demonstrating that this generalization remains dependent on the structure of the training distribution. Overall, the results provide evidence that NAR can offer a favorable trade-off between solution quality and computational efficiency for large facility-location instances.

## 1 Introduction

The field of operations research focuses on algorithms that are often applied in the design of real-world problems such as logistics routing, job shop scheduling etc. [Petropoulos et al. \[2024\]](#) which all have a significant impact on the business or policy strategy. Benefits gained by good algorithms often scales with the problem size, as the potential of improvement increases. One common issue is that many such optimization tasks are discrete-combinatorial in nature, meaning that the decision variables are constrained to a binary/integer domain. This makes precise optimization much more difficult [Petropoulos et al.](#)

[\[2024\]](#), as methods which are based on search within continuous space lose their optimum guarantees.

Discrete optimization problems arise in many applications of operations research, including logistics, scheduling and resource allocation [Petropoulos et al. \[2024\]](#). One representative problem is the uncapacitated facility location problem (UFLP), in which a subset of candidate facilities must be opened and every client assigned to an opened facility. The objective is to minimize the combined facility-opening and client-connection costs. Exact mixed-integer solvers can find globally optimal solutions, but their computational cost increases rapidly with the size of the problem. Approximation algorithms provide more scalable alternative, although this scalability is obtained by relaxing the requirement of optimality.

Machine learning has increasingly been applied to combinatorial optimization, with graph neural networks being particularly suitable for problems whose structure can be represented relationally [Cappart et al. \[2022\]](#). Neural Algorithmic Reasoning (NAR) follows a related approach by training neural networks to reproduce the intermediate computation of an existing algorithm rather than learning only a direct mapping from an input to a final solution [Veličković and Blundell \[2021\]](#). Recent work has extended this idea to primal-dual approximation algorithms for NP-hard problems, whose intermediate variables and dependencies provide a natural structure for graph-based processing [He and Vitercik \[2025\]](#).

This work applies NAR to the UFLP using the Jain-Vazirani primal-dual approximation algorithm as the teacher [Jain and Vazirani \[1999\]](#). A custom encoder-processor-decoder architecture is designed to follow the information flow of the teacher algorithm and is trained using both intermediate execution traces and final solution-cost supervision. The model is evaluated along three dimensions: generalization to problem sizes beyond those encountered during training, robustness to changes in the connection-cost distribution and solution-time scaling. The experiments examine whether the learned procedure can retain useful algorithmic behavior beyond the specific instances and assumptions represented in its training data.

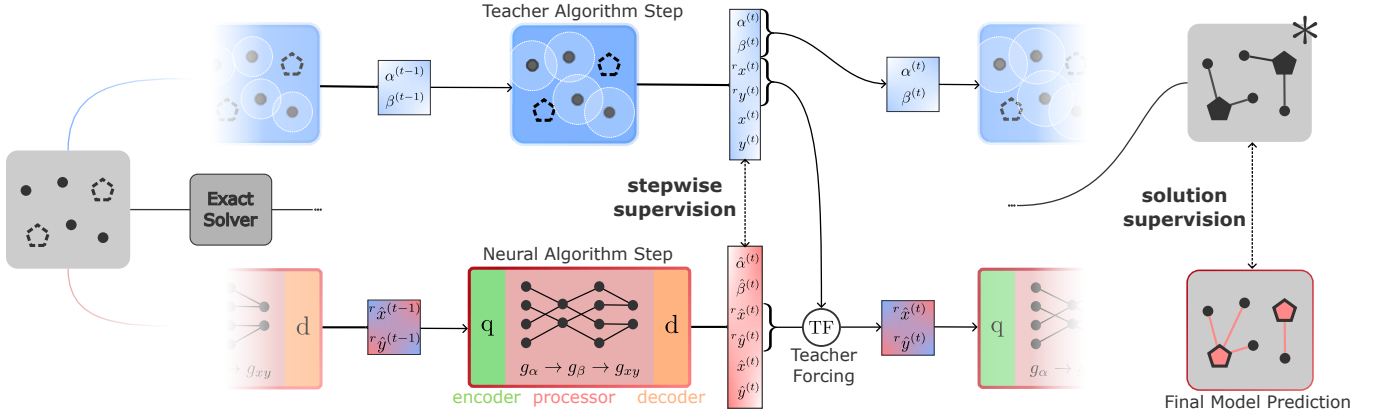


Figure 1: Overview of the NAR training and inference pipeline. The Jain–Vazirani teacher algorithm generates the next trace state from the state at iteration (t-1) (upper branch), while the NAR model predicts the same step using an encoder  $q$ , the sequential processor  $g_\alpha \rightarrow g_\beta \rightarrow g_{xy}$ , and a decoder  $d$  (lower branch). The decoded algorithm variables receive stepwise supervision from the teacher trace, while teacher forcing determines whether teacher or predicted residual states are propagated to the next iteration. After the rollout, the predicted solution cost is supervised against the optimum computed independently by the exact solver. During inference, the model is rolled out without teacher traces, teacher forcing, or exact-solver supervision.

## 2 Background

### 2.1 Facility Location Optimization

One exemplar problem is the facility location problem (FLP) [He and Vitercik \[2025\]](#). The optimization revolves around 2 sets of inputs, the client locations and the potential facility locations. The goal is to select at which locations facilities should be built, in order to find an optimum in a given objective, such as minimizing the traveling costs, minimizing the maximal distance to clients etc. There are variations of the FLP problem depending on the objectives and constraints. Within this project, the specific version in focus is the Uncapacitated Facility Location Optimization. The main characteristics of this problem are as follows:

- **Infinite Capacity:** There is no upper bound on the number of clients assigned to a single facility.
- **Unconstrained Siting:** There is no fixed limit on the total number of facilities that can be established.
- **Cost Minimization:** The primary objective is the minimization of the joint cost function, comprising both aggregate client traveling costs and fixed facility activation costs.

### 2.2 Exact solvers

The uncapacitated facility location problem can be formulated as a binary mixed-integer linear program. Let  $y_j$  indicate whether candidate facility  $j$  is opened, and let  $x_{ij}$  indicate whether client  $i$  is assigned to facility  $j$ . Given facility opening costs  $f_j$  and assignment costs  $d_{ij}$ ,

the problem is:

$$\min_{x,y} \sum_j f_j y_j + \sum_i \sum_j d_{ij} x_{ij},$$

such that

$$\sum_j x_{ij} = 1 \forall i, \quad x_{ij} \leq y_j \forall i, j, \quad x_{ij}, y_j \in \{0, 1\}$$

The first constraint assigns every client to exactly one facility, while the second ensures that assignment only happens with opened facilities. Mixed-integer solvers solve this formulation using various methods such as linear programming relaxations, branch-and-bound and cutting planes. Once the solver closes the optimality gap, the resulting solution is guaranteed to be globally optimal.

This standard formulation contains  $IJ$  assignment variables and  $IJ$  linking constraints for the set of clients  $I$  and the set of potential locations  $J$ . The formulation size therefore grows as  $\mathcal{O}(IJ)$ , while the mixed-integer solution procedure can exhibit exponential worst-case run-time growth.

### 2.3 Approximation algorithms

When exact optimization becomes computationally expensive, approximation algorithms provide an alternative by trading optimality for computational efficiency. Unlike general heuristics, approximation algorithms provide theoretical guarantees on the quality of the returned solution.

A prominent approach for the metric uncapacitated facility location problem is the primal-dual approximation algorithm of Jain and Vazirani [Jain and Vazirani \[1999\]](#).

---

**Algorithm 1:** High-level Jain–Vazirani primal-dual algorithm for metric UFLP

---

**Input:** Clients  $I$ , facilities  $J$ , opening costs  $f_j$ , connection costs  $d_{ij}$

**Output:** Opened facilities  $F \subseteq J$  and client assignments

Initialize  $\alpha_i \leftarrow 0$  for all  $i \in I$ ;

Initialize  $\beta_{ij} \leftarrow 0$  for all  $(i, j) \in I \times J$ ;

Initialize all clients as *active*;

Initialize  $F \leftarrow \emptyset$ ;

**while** *there exists an active client* **do**

    Uniformly increase  $\alpha_i$  for all active clients  $i$ ;

**foreach** *active client*  $i$  **and** *facility*  $j$  **do**

**if**  $\alpha_i \geq d_{ij}$  **then**

$\beta_{ij} \leftarrow \alpha_i - d_{ij}$ ;

**foreach** *facility*  $j$  **do**

**if**  $\sum_i \beta_{ij} \geq f_j$  **then**

            Mark facility  $j$  as *tight*;

**foreach** *tight facility*  $j$  **do**

        Open facility  $j$  tentatively;

**foreach** *active client*  $i$  **with**  $\alpha_i \geq d_{ij}$  **do**

            Mark client  $i$  as served and deactivate it;

Apply a pruning / reverse-delete step to remove redundant tentatively opened facilities;

Assign each client to one opened facility;

**return**  $F$  and the resulting assignments

---

The method operates on the linear programming relaxation of the facility location problem and its dual, constructing a feasible facility opening and client assignment solution while using the dual variables to guide the algorithm.

For the metric setting, the Jain-Vazirani (JV) algorithm provides a 3-approximation guarantee. The algorithm additionally admits an efficient combinatorial implementation, making it suitable for solving substantially larger instances than those for which exact optimization may be practical.

The JV algorithm proceeds by iteratively increasing dual variables for active clients, which are used as a “funding” mechanism for facilities. Facilities are opened once their total received contributions are greater or equal to their opening costs. A high level overview of the algorithm is shown in Algorithm 1. This iterative primal-dual structure is particularly suitable for Neural Algorithmic Reasoning, as it exposes intermediate states that can be represented explicitly and learned by a graph-based processor.

## 2.4 Machine learning approaches

Machine learning has increasingly been applied to combinatorial optimization problems, either by supporting

existing optimization algorithms or by directly predicting solutions Cappart et al. [2022]. In solver assisted approaches, learned models can be used to guide decisions within exact or heuristic algorithms, by for example branching variables, predicting promising solutions or providing warm starts. Alternatively, end-to-end approaches attempt to directly map a problem instance to a feasible or near-optimal solution.

Graph neural networks (GNNs) are particularly suitable for many combinatorial optimization problems, since problem instances can naturally be represented as graphs whose nodes and edges correspond to decision variables, constraints, entities or interactions between them Cappart et al. [2022]. The graph aggregation also enables the GNN based models to operate on instances of varying size without requiring a fixed-dimensional input representation.

A major challenge for learned combinatorial optimization methods is generalization beyond the distribution and problem sizes encountered during training. This is particularly relevant when training labels are obtained using exact optimization algorithms, since generating such labels for large instances can itself become computationally expensive. Consequently, methods capable of learning on relatively small instances while generalizing to larger ones are of particular interest.

## 2.5 Neural Algorithmic Reasoning

Neural Algorithmic Reasoning (NAR) aims to train neural networks to reproduce the computation performed by classical algorithms Veličković and Blundell [2021]. Rather than learning only a mapping from a problem instance to its final solution, the model is trained to imitate the intermediate states and operations of an algorithm.

A common NAR architecture follows an encode-process-decode structure. The encoder maps algorithm inputs and intermediate states into latent representations. A processor repeatedly operates on these representations to approximate the computational steps of the target algorithm. The decoder maps the resulting latent states back into the algorithm’s output space. The processor is commonly implemented using a graph neural network.

Supervising intermediate algorithm steps provides a stronger learning signal than supervising only the final output and encourages the processor to learn the computational structure of the underlying algorithm Veličković et al. [2022]. An important objective of NAR is therefore algorithmic generalization: if the learned processor captures the rules followed by an algorithm rather than statistical properties specific to the training instances, it may remain applicable to problem instances that are larger than those observed during training.

Applying NAR to NP-hard combinatorial optimization problems introduces an additional difficulty compared to classical algorithms, since efficient exact algorithms that can serve as teacher algorithms are gener-

ally unavailable. He and Vitercik [He and Vitercik 2025] propose using primal-dual algorithms to bridge this gap. The primal-dual paradigm provides efficient approximation algorithms for a range of NP-hard problems, while the interaction between primal and dual variables admits a graph representation that aligns naturally for extending NAR from polynomial-time algorithmic tasks to NP-hard combinatorial optimization problems.

Following this approach, this work applies NAR to the uncapacitated facility location problem using the Jain-Vazirani primal-dual approximation algorithm as the teacher algorithm. The following section describes how the computational structure of JV is represented within the NAR architecture.

## 3 Methodology

### 3.1 Traces from teacher algorithm

To train the NAR model, the JV algorithm is used to generate training data by recording its execution traces, i.e. the values of selected variables at each algorithmic step. These traces include the variables appearing in Algorithm 1, in particular  $\alpha^{(t)}$  and  $\beta^{(t)}$  for each step  $t$ .

To provide additional supervision, the scalar  $\delta \in \mathbb{R}$  is also recorded. It is defined as

$$\delta^{(t)} \leftarrow \alpha_*^{(t)} - \alpha_*^{(t-1)},$$

and represents the amount by which the active  $\alpha$  variables are uniformly increased during step  $t$  until at least one associated constraint becomes tight.

In addition, two residual variables are tracked to explicitly represent quantities that are otherwise implicit in the JV algorithm. The client connection residual

$$r_{x_{ij}}^{(t)} \leftarrow d_{ij} - \alpha_i^{(t)}$$

represents the discrepancy between the current client budget  $\alpha_i^{(t)}$  and the connection cost  $d_{ij}$ . It therefore indicates how much further the client’s budget must increase before becoming tight with candidate facility  $j$ . The facility residual

$$r_{y_j}^{(t)} \leftarrow f_j - \sum_i \beta_{ij}^{(t)}$$

represents the portion of the facility  $j$ ’s opening cost that remains uncovered by the contributions received from clients. Together, these residuals provide an explicit representation of the evolving state of the JV algorithm that can be used as input and supervision for the NAR model.

### 3.2 Encoder

At each algorithmic step  $t$ , the processor receives the residual states  $r_{x^{(t-1)}}$  and  $r_{y^{(t-1)}}$ . Two separate encoders,  $q_x$  and  $q_y$ , map these scalar residual values into

the common  $d_h$ -dimensional latent space used by the processor:

$$h_{r_x}^{(t-1)} = q_x(r_{x^{(t-1)}}), \quad h_{r_y}^{(t-1)} = q_y(r_{y^{(t-1)}}).$$

Where  $q_x$  and  $q_y$  are linear layers followed by a Layer-Norm and PReLU activation.

### 3.3 Processor

For each algorithmic iteration, the processor performs three sequential message passing phases,  $g_\alpha$ ,  $g_\beta$  and  $g_{xy}$ . The three phases are designed to reflect the dependency structure of the variables observed in the JV traces. A complete sequence  $g_\alpha \rightarrow g_\beta \rightarrow g_{xy}$  corresponds to one predicted iteration of the teacher algorithm.

As stated before, the processor phases are GNN operations. The graph on which these phases operate is constructed of nodes that reflect the JV algorithm variables. There are 4 types of nodes,  $r_{x_{ij}}$ ,  $r_{y_j}$ ,  $\alpha_i$  and  $\beta_{ij}$ . The edges between nodes are then defined by their shared indices. For example,  $r_{x_{ij}}$  and  $\alpha_i$  are connected based on sharing the same  $i$ . Each of the individual GNN phases uses only a directed subset of the nodes and edges.

#### 3.3.1 $g_\alpha$ step

During the first message passing phase, the  $\alpha_i$  nodes receive messages from the  $r_{x_{ij}}$  nodes. Specifically, for every client  $i$ , the  $\alpha_i$  node receives information from all the residual nodes  $r_{x_{ij}}, j \in J$  associated with that client. The message uses a combination of 2 aggregation methods: the *min* (denoted by  $\downarrow$ ) and the *sum* (denoted by  $\Sigma$ ). These are selected as the *min* should give information about any tight connections, while the *sum* holds more general information.

$$h_\alpha^{(t)} = \text{SAGEConv}(\downarrow h_{r_x}^{(t-1)}, \Sigma h_{r_x}^{(t-1)})$$

#### 3.3.2 $g_\beta$ step

During the second message passing phase, information is passed from the  $\alpha_i$  and  $r_{y_j}$  nodes to form the hidden representation of  $\beta_{ij}$  nodes. A  $\beta_{ij}$  node has an edge with  $\alpha_i$  sharing the same  $i$ , and an edge with a  $r_{y_j}$  sharing the same  $j$ . Each of these edges constitute a 1-to-1 connection, thus no aggregation mechanism is used.

$$h_\beta^{(t)} = \text{SAGEConv}(h_\alpha^{(t)}, h_{r_y}^{(t-1)})$$

#### 3.3.3 $g_{xy}$ step

During the third and final message passing phase, information is propagated back to the  $r_x$  and  $r_y$  nodes, producing the updated hidden representations for the current algorithmic iteration.

For each  $r_{x_{ij}}$  node, messages are received from the corresponding  $\alpha_i$  and  $\beta_{ij}$  nodes. Both of these connections are 1-to-1, therefore no aggregation is used. The  $\alpha_i$  should

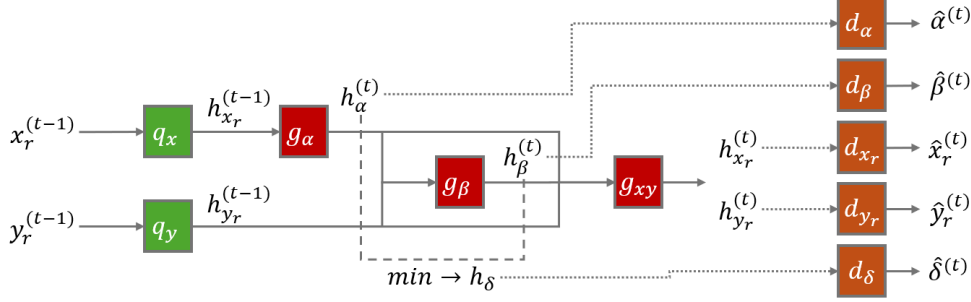


Figure 2: Overview of the NAR structure and its components across the dual-column layout. Green squares denote the encoders, red squares are the processor steps. Lastly the decoders are denoted as orange.

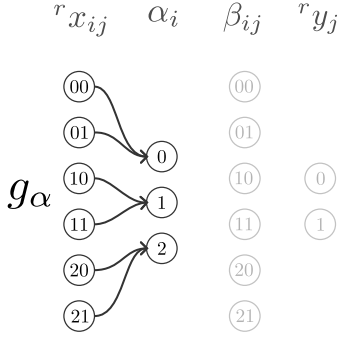


Figure 3: Graph connectivity during the  $g_\alpha$  message passing step. Example for a problem with 3 client locations and 2 potential facility locations

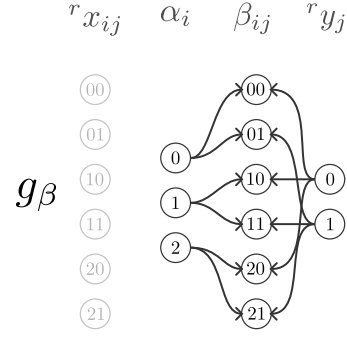


Figure 4: Graph connectivity during the  $g_\beta$  message passing step. Example for a problem with 3 client locations and 2 potential facility locations

carry information about the budget contribution of the given client, while  $\beta_{ij}$  carries information about potential facility opening cost tightness.

$$h_{x_r}^{(t)} = \text{SAGEConv}(h_\alpha^{(t)}, h_\beta^{(t)})$$

The facility residual nodes  $^r y_j$  receive information from all  $\beta_{ij}$  nodes associated with facility  $j$ . In addition, information from all  $\alpha_i$  is globally pooled using both *min* and *sum* aggregation. The  $\beta_{ij}$  messages provide facility specific information, while the aggregated  $\alpha_i$  representations provide information of the overall state of the client variables.

$$h_{y_r}^{(t)} = \text{SAGEConv}(\downarrow h_\alpha^{(t)}, \Sigma h_\alpha^{(t)}, h_\beta^{(t)})$$

These three phases  $g_\alpha \rightarrow g_\beta \rightarrow g_{xy}$  transform the residual state from the preceding iteration into updated hidden representation corresponding to one predicted step of the JV algorithm.

### 3.4 Decoder

After all three phases of the processor, the hidden states of all nodes is decoded back into the original algorithm dimension. This is done at each algorithmic step to provide

strong supervision. The decoded residuals

$$^r \hat{x}^{(t)} = d_{x_r}(h_{x_r}^{(t)})$$

$$^r \hat{y}^{(t)} = d_{y_r}(h_{y_r}^{(t)})$$

are used for both for supervision and propagated to the next step of the algorithm, subject to replacement via teacher forcing. The remaining decoded values are purely for supervision purposes:

$$\hat{\alpha}^{(t)} = d_\alpha(h_\alpha^{(t)}) \quad \hat{\beta}^{(t)} = d_\beta(h_\beta^{(t)})$$

$$\hat{x}^{(t)} = \tilde{d}_x(h_{x_r}^{(t)}) \quad \hat{y}^{(t)} = \tilde{d}_y(h_{y_r}^{(t)})$$

$$\hat{\delta}^{(t)} = d_\delta(\downarrow(h_\alpha^{(t)}, h_\beta^{(t)}))$$

which includes node-by-node prediction on the algorithm variables  $\alpha$  and  $\beta$  as well as  $\delta$ , which uses the min aggregation over the hidden states of the  $h_\alpha$  and  $h_\beta$  nodes. The primal logits are also predicted as  $\hat{x}^{(t)}$  and  $\hat{y}^{(t)}$ .  $\delta$  is the amount by which the algorithm step uniformly increases the client budget contribution until any of the dual constraints become tight, thus motivating the use of the *min* aggregation.

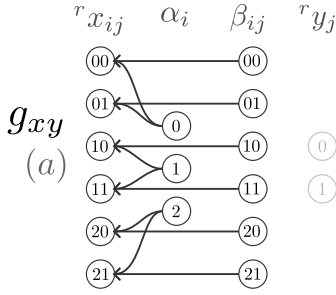


Figure 5: Graph connectivity during the  $g_{xy}$  message passing step, specifically when the part directed into  $x$  nodes. Example for a problem with 3 client locations and 2 potential facility locations

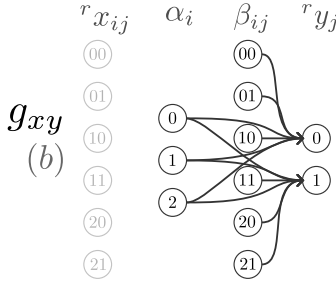


Figure 6: Graph connectivity during the  $g_{xy}$  message passing step, specifically when the part directed into  $y$  nodes. Example for a problem with 3 client locations and 2 potential facility locations

The decoders used are as follows:

$$\begin{aligned} d_{\bullet}(\bullet) &= PReLU(W \bullet + b) \\ \tilde{d}_{\bullet}(\bullet) &= PW \bullet + b \end{aligned}$$

where the  $PReLU$  activation function is used for the variables that should remain positive ( $r\hat{x}, \hat{\alpha}, \hat{\beta}, r\hat{y}, \hat{\delta}$ ) and raw logits are returned for the  $\hat{x}, \hat{y}$ , which are used as direct predictors for the primal variables.

### 3.5 Inference

The primal decoders  $d_x$  and  $d_y$  produce logits  $\hat{x}^{(t)}$  and  $\hat{y}^{(t)}$ . During training, these logits are used in the primal supervision and objective value loss, by using a *Sigmoid* activation. After training is finished, inference steps use the residual variables  $r\hat{x}^{(t)}$  and  $r\hat{y}^{(t)}$  to predict facility opening and client connection according to the following rules:

$$\begin{aligned} \bar{y}_j &= \begin{cases} 1, & \text{if } r\hat{y}_j \leq 0 \\ 0, & \text{otherwise} \end{cases} \\ \bar{x}_{ij} &= \begin{cases} 1, & \text{if } \hat{y}_j = 1 \wedge r\hat{x}_{ij} \leq 0 \\ 0, & \text{otherwise} \end{cases} \end{aligned}$$

Once a client  $i$  is served, its associated  $r x_{ij}$ ,  $\alpha_i$  and  $\beta_{ij}$  nodes and incident edges are masked from subsequent processor steps. Inference stops when all clients have been removed from the processor graph, i.e. are marked as served.

Following the pruning/reverse-delete step of Algorithm 1, after all clients have been assigned a small repair step is taken. The repair step reconnects each client to the nearest facility which has been opened by the NAR model, to repair any early connections to facilities which could later be improved by other newly opened facilities. Subsequently, any facilities that have no clients assigned after the repair step are closed.

### 3.6 Loss function

To ensure correct supervision, during training the NAR was always made as many predictive steps as the JV algorithm. Different losses are computed at each step using the teacher algorithm, specifically:

$$\begin{aligned} \mathcal{L}_{\alpha}^{(t)} &= \text{MSE}(\hat{\alpha}^{(t)}, \alpha^{(t)}) & \mathcal{L}_{\beta}^{(t)} &= \text{MSE}(\hat{\beta}^{(t)}, \beta^{(t)}) & (1) \\ \mathcal{L}_{\delta}^{(t)} &= \text{MSE}(\hat{\delta}^{(t)}, \delta^{(t)}) & \mathcal{L}_{opt} &= \text{SL1}(c(\hat{x}, \hat{y}), c(x^*, y^*)) & (2) \end{aligned}$$

$$\begin{aligned} \mathcal{L}_{r_x}^{(t)} &= \text{MSE}(r\hat{x}^{(t)}, r x^{(t)}) & \mathcal{L}_{r_y}^{(t)} &= \text{MSE}(r\hat{y}^{(t)}, r y^{(t)}) & (3) \\ \mathcal{L}_x^{(t)} &= \text{BCE}(\hat{x}^{(t)}, x^{(t)}) & \mathcal{L}_y^{(t)} &= \text{BCE}(\hat{y}^{(t)}, y^{(t)}) & (4) \end{aligned}$$

where SL1 denotes the Smooth  $L_1$  loss calculated against the exact optimum  $(x^*, y^*)$  upon completion of all  $T$  steps.

The full loss function is then:

$$\begin{aligned} \mathcal{L} &= \lambda_{opt} \mathcal{L}_{opt} + \sum_{t=0}^T (\lambda_{\alpha} \mathcal{L}_{\alpha}^{(t)} + \lambda_{\beta} \mathcal{L}_{\beta}^{(t)} + \lambda_{\delta} \mathcal{L}_{\delta}^{(t)} + \\ &\quad \lambda_{r_x} \mathcal{L}_{r_x}^{(t)} + \lambda_{r_y} \mathcal{L}_{r_y}^{(t)} + \lambda_x \mathcal{L}_x^{(t)} + \lambda_y \mathcal{L}_y^{(t)}) & (5) \end{aligned}$$

where  $\lambda_i$  represent the hyperparameter coefficients for each loss term.

## 4 Experiments

The experiments are designed to evaluate generalization along two dimensions: problem size and distance structure. Problem size generalization measures whether a model trained on small instances can be applied to substantially larger ones without a dramatic decrease in performance. Distance structure generalization measures whether the learned reasoning procedure remains applicable when some details (here the connection cost calculation) are altered.

### 4.1 Dataset generation

An instance with  $I$  clients and  $J$  candidate facilities is denoted by  $(I, J)$ . Client coordinates  $\mathbf{c}_i \in \mathbb{R}^2$ , facility

coordinates  $\mathbf{f}_j \in \mathbb{R}^2$  and facility opening costs  $o_j$  are sampled independently according to

$$\begin{aligned}\mathbf{c}_i &\sim \mathcal{N}(\mathbf{0}, \mathbf{I}_2)^{I \times 2} \\ \mathbf{f}_j &\sim \mathcal{N}(\mathbf{0}, \mathbf{I}_2)^{J \times 2} \\ o_j &\sim \mathcal{U}(0, 1)^J\end{aligned}$$

The coordinates are used solely to construct the connection-cost matrix. The model receives only the resulting matrix and the facility opening costs rather than the raw coordinates.

#### 4.1.1 Distance structures

Three methods are used to construct the client-facility connection cost matrix.

**Metric distances.** For the metric dataset, each connection cost is the Euclidean distance between the generated client and facility coordinates:

$$d_{ij} = \sqrt{\mathbf{c}_i - \mathbf{f}_j}$$

This connection cost calculation preserves the metric structure assumed by the JV teacher algorithm. Metric instances are used as the in-distribution setting during training and validation.

**Weighted distances.** For the weighted dataset, a single "demand" scalar weight is independently generated for each client:

$$w_i \sim \mathcal{U}[0, 1]$$

The metric connection costs are then weighted as

$$w_i D_{ij}^{\text{metric}}$$

Introduction of the demand weights changes the relative contribution of different clients to the objective and generally no longer represents distances within a shared Euclidean metric space. The weighted dataset therefore introduces a distribution shift with respect to the metric instances.

**Random distances.** For the random dataset, the metric distance matrix is discarded and replaced by an independently sampled random matrix:

$$D_{ij}^{\text{random}} \sim \mathcal{U}[0, 1]$$

The resulting connection costs have no relationship to the generated coordinates and do not preserve Euclidean or metric structure. This dataset has the role of the strongest geometry shift considered in the experiments.

Facility opening costs follow the same distribution under all three constructions. Since weighted and random distance structures are used only at test time, performance on these datasets is an evaluation on the out-of-distribution generalization.

#### 4.1.2 Problem sizes

The training set contains 2250 problem instances across three sizes configurations:

$$(10, 10) : 1500, \quad (5, 10) : 375, \quad (10, 5) : 375$$

The validation set is generated independently using metric instances with sizes up to (100, 100). It is used for model selection and hyperparameter evaluation without exposing the model to alternative distance structures.

$$\begin{aligned}(10, 10) : 200, & \quad (5, 10) : 125, & \quad (10, 5) : 125, \\ (15, 15) : 100, & \quad (20, 20) : 50, & \quad (30, 30) : 30 \\ & & \quad (100, 100) : 15\end{aligned}$$

The test set is generated with even larger instances, and across the 3 different distance structures.

$$\begin{aligned}(5, 5) : 20, & \quad (10, 10) : 20, & \quad (15, 15) : 20, & \quad (20, 20) : 20, \\ (10, 20) : 20, & \quad (20, 10) : 20, & \quad (30, 30) : 20, & \quad (50, 50) : 20, \\ (100, 100) : 20, & \quad (200, 200) : 10, & \quad (300, 300) : 5, \\ & \quad (400, 400) : 5, & \quad (500, 500) : 5.\end{aligned}$$

## 4.2 Experiment Setup

The model was initiated with a hidden dim of 128, leading to a total number of parameters 660,494. The model was trained over 200 epochs. The loss function used was the one mentioned before, with arbitrarily selected coefficients:

$$\begin{aligned}\mathcal{L} = 0.1\mathcal{L}_{\text{opt}} + \sum_{t=0}^T & (0.5\mathcal{L}_{\alpha}^{(t)} + 0.5\mathcal{L}_{\beta}^{(t)} + 1\mathcal{L}_{\delta}^{(t)} + \\ & 2\mathcal{L}_{rx}^{(t)} + 2\mathcal{L}_{ry}^{(t)} + 0.5\mathcal{L}_x^{(t)} + 0.5\mathcal{L}_y^{(t)})\end{aligned}\quad (6)$$

To account for the possibility of having multiple different good solutions (which would get penalized by the losses based on teacher algorithm) the coefficients for  $\mathcal{L}_{\alpha}, \mathcal{L}_{\beta}, \mathcal{L}_{rx}, \mathcal{L}_{ry}, \mathcal{L}_x, \mathcal{L}_y, \mathcal{L}_{\delta}$  loss terms are set to linearly decrease and become 0 at epoch 150. The only remaining loss term after epoch 150 is then the  $\mathcal{L}_{\text{opt}}$ . To further help with stability, a teacher forcing method was used [Veličković et al. \[2022\]](#). The teacher forcing probability was also linearly decreased and becomes 0 at epoch 150.

## 5 Results

The NAR model is evaluated against both the JV teacher algorithm and the exact optimum. Solution quality is reported as the ratio between the objective value of either the teacher algorithm or the NAR model and the exact optimum for the same instance. A ratio of 1 therefore represents an optimal solution. The curves in Figures 7-10 show the median ratio across instances of increasing problem sizes.

## 5.1 Problem size generalization

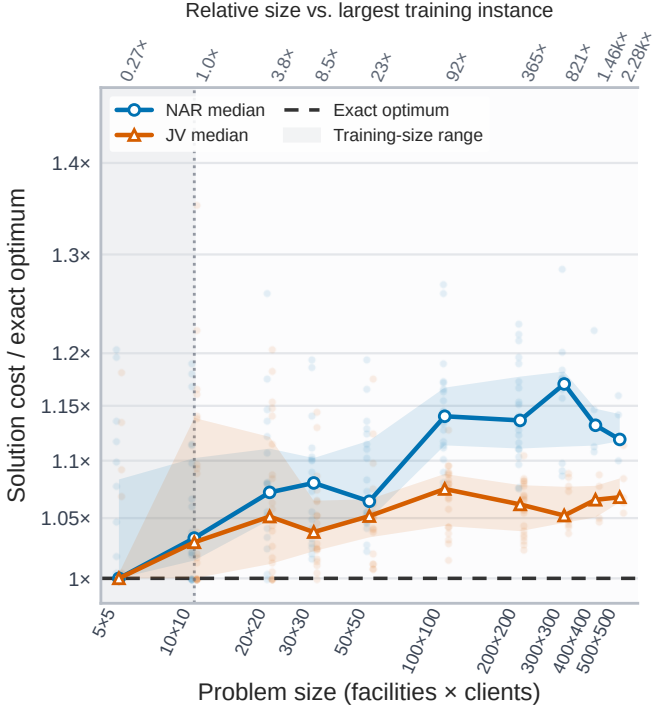


Figure 7: Solution quality on metric test instances of increasing size. Objective values are normalized by the exact optimum, such that (1) corresponds to an optimal solution and lower values are better. The curves show the median performance of the NAR model and the Jain–Vazirani (JV) algorithm, faint points represent individual test instances, and the shaded bands summarize the variation across instances. The gray region and vertical dotted line indicate the range of instance sizes represented during training. The upper axis reports the number of decision variables relative to the largest training configuration.

Figure 7 evaluates the problem size generalization while retaining the metric distance structure used during training. Within the training-size range, the NAR and JV solutions have similar objective values. As the problem size increases, the JV solutions have a better median solution quality, although the NAR model does outperform the teacher algorithm on certain instances. Importantly, the quality of the NAR solutions does not blow up with problem size increased up to a size of approximately  $2280\times$  the size that the model has seen during training. Both solutions still remain relatively close to the optimal solution.

Further scaling trends have been evaluated purely between the JV algorithm and the NAR model, since finding the optimal solution becomes much more computationally expensive at these sizes. As can be seen in Figure 8, even as instances approach sizes that are approximately  $9100\times$  larger than the largest training instances, the NAR solution costs scale approximately in parallel with the JV algorithm solutions. While comparison with the optimum

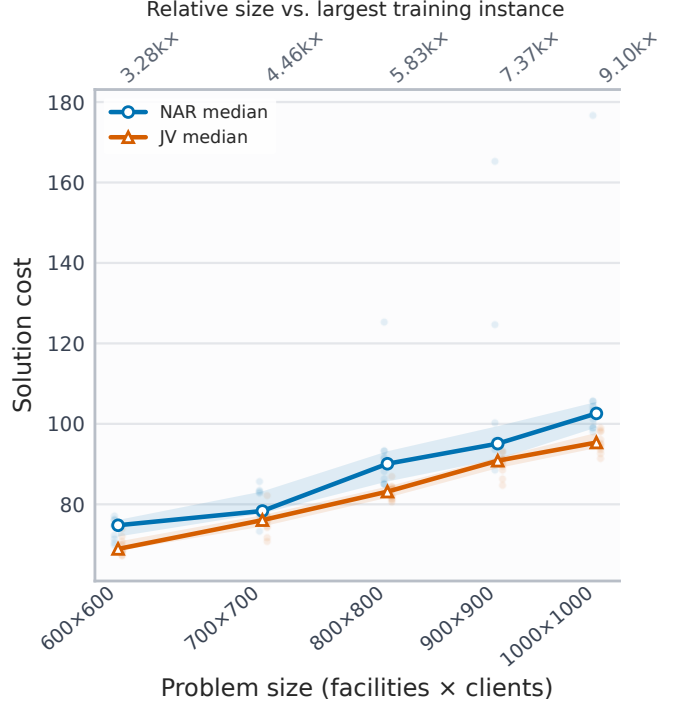


Figure 8: Solution costs on large metric test instances for which exact optima were not computed. The curves show the median objective values returned by the NAR model and the Jain–Vazirani (JV) algorithm for configurations from (600, 600) to (1000, 1000); faint points represent individual instances, and the shaded bands summarize the variation across instances. The upper axis reports the number of decision variables relative to the largest training configuration, (10, 10).

is not available, the 3-approximation guarantee of the JV algorithm gives some context to the quality of the NAR model solutions. These results provide evidence of size generalization within the metric data distribution: the model remains applicable far beyond the sizes observed during training, although it does not match the solution quality of its teacher algorithm.

## 5.2 Distance-structure generalization

To evaluate whether the learned neural algorithm procedure depends on the metric structure encountered during training, the same model is tested on demand weighted and fully random connection costs. Neither distance structure is included in the training or validation data.

### 5.2.1 Demand-weighted distances

Figure 9 shows that the NAR model remains close to the exact optimum when the metric distances are weighted by independently sampled client demands. In contrast, the performance of the JV solutions deteriorates as the problem size increases, reaching more than four times the optimal cost on the largest instances.

Weighting the distances by client-specific values generally violates the metric assumptions under which the approximation guarantee of JV is derived. The degradation of JV is therefore expected and does not contradict its guarantee for metric UFLP. Nevertheless, the NAR model was trained only on traces produced by JV on metric instances. Its substantially better performance under this distribution shift suggests that it has not merely reproduced the teacher’s final decisions. Instead, the learned procedure retains useful behavior under this particular modification of the connection-cost structure. This conclusion is limited to the tested weighting scheme and does not imply robustness to arbitrary non-metric costs.

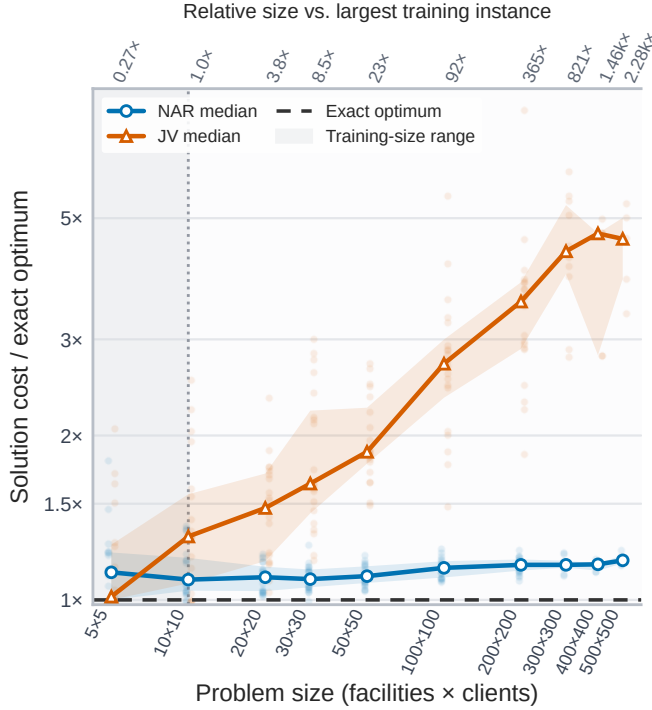


Figure 9: Solution quality on out-of-distribution instances with demand-weighted connection costs. The model is trained only on unweighted metric instances. Objective values are normalized by the exact optimum, and lower values are better. The NAR approximation ratio remains comparatively stable as the instances grow, whereas the performance of JV deteriorates because the weighted costs no longer necessarily satisfy its metric assumptions. Plot elements follow the notation of Figure 7.

### 5.2.2 Random distances

The fully random distance matrices in Figure 10 constitute a stronger distribution shift because the connection costs no longer contain any information derived from the generated coordinates. Both methods consequently deteriorate as the instances grow. The NAR model remains relatively close to the optimum on the smaller configurations, but its median approximation ratio begins to in-

crease noticeably for larger instance sizes.

JV deteriorates at smaller problem sizes, and the NAR model produces lower median costs than JV for most intermediate configurations. However, this advantage disappears on the largest instances, where the NAR median becomes higher than the JV median. Thus, the model retains some useful behavior under the random-distance shift, particularly at moderate problem sizes, but it does not generalize uniformly when a severe change in distance structure is combined with a large increase in problem size.

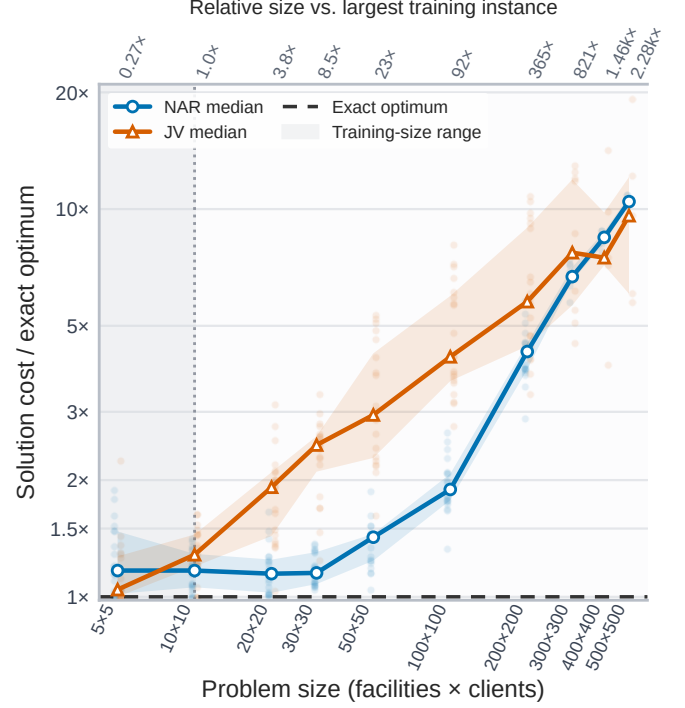


Figure 10: Solution quality on out-of-distribution instances with independently sampled random connection costs. These costs are unrelated to the generated coordinates and represent the strongest distance-structure shift considered. Both NAR and JV deteriorate with increasing problem size. NAR performs better than JV across most intermediate sizes, but this advantage disappears on the largest configurations. Plot elements follow the notation of Figure 7.

## 5.3 Solution-time scaling

The median solution times for the metric, demand-weighted, and random-distance datasets are reported in Tables 1-3. The experiments containing configurations up to (500, 500) were performed on a laptop equipped with an NVIDIA RTX 3070 Laptop GPU, an AMD Ryzen 7 5800H processor, and 32GB of DDR4 memory. The additional metric scaling experiments from (600, 600) to (1000, 1000) were performed using a Runpod PyTorch 2.8.0 instance equipped with an NVIDIA RTX 3090 with

Table 1: Median solution time in seconds for metric-distance instances. The  $600 \times 600$ – $1000 \times 1000$  scale rows were collected on different hardware than the smaller datasets; the exact solver was not run.

| Instance           | Median time (s) |         |        |
|--------------------|-----------------|---------|--------|
|                    | NAR             | JV      | Exact  |
| $5 \times 5$       | 0.056           | 0.00080 | 0.0048 |
| $10 \times 10$     | 0.075           | 0.0017  | 0.0054 |
| $20 \times 20$     | 0.099           | 0.0056  | 0.0093 |
| $30 \times 30$     | 0.105           | 0.011   | 0.014  |
| $50 \times 50$     | 0.124           | 0.029   | 0.036  |
| $100 \times 100$   | 0.143           | 0.108   | 0.220  |
| $200 \times 200$   | 0.236           | 0.463   | 1.90   |
| $300 \times 300$   | 0.420           | 1.23    | 7.08   |
| $400 \times 400$   | 0.660           | 2.36    | 16.1   |
| $500 \times 500$   | 1.06            | 4.29    | 25.8   |
| $600 \times 600$   | 0.749           | 6.49    |        |
| $700 \times 700$   | 1.05            | 10.5    |        |
| $800 \times 800$   | 1.38            | 12.5    |        |
| $900 \times 900$   | 1.69            | 15.4    |        |
| $1000 \times 1000$ | 2.03            | 20.6    |        |

Table 2: Median solution time in seconds for random-distance instances.

| Instance         | Median time (s) |         |        |
|------------------|-----------------|---------|--------|
|                  | NAR             | JV      | Exact  |
| $5 \times 5$     | 0.036           | 0.00080 | 0.0048 |
| $10 \times 10$   | 0.037           | 0.0018  | 0.0062 |
| $20 \times 20$   | 0.044           | 0.0054  | 0.0099 |
| $30 \times 30$   | 0.044           | 0.011   | 0.020  |
| $50 \times 50$   | 0.046           | 0.032   | 0.070  |
| $100 \times 100$ | 0.046           | 0.125   | 0.814  |
| $200 \times 200$ | 0.083           | 0.503   | 7.79   |
| $300 \times 300$ | 0.156           | 1.25    | 153    |
| $400 \times 400$ | 0.258           | 2.59    | 172    |
| $500 \times 500$ | 0.371           | 4.98    | 1084   |

24,GB of VRAM, 125,GB of system memory, and 32 virtual CPUs, with CUDA 13.0 available. Because these groups were evaluated on different hardware, their absolute runtimes should not be compared directly.

For small instances, the computational overhead of NAR inference causes it to be slower than both JV and the exact solver. As the problem size increases, however, the NAR solution time grows considerably more slowly than that of exact optimization. On the metric dataset, the median time at  $(500, 500)$  is 1.06 seconds for NAR, compared with 25.8 seconds for the exact solver. The difference is larger on the demand-weighted dataset, where the corresponding times are 1.20 and 52.5 seconds. The strongest difference occurs for the random-distance instances: at  $(500, 500)$ , NAR requires 0.371 seconds, whereas the exact solver requires 1084 seconds. NAR therefore becomes approximately 24, 44, and 2900 times faster than exact optimization on the largest metric, demand-weighted, and random-distance instances, respectively.

NAR also eventually becomes faster than the JV im-

Table 3: Median solution time in seconds for weighted-distance instances.

| Instance         | Median time (s) |         |        |
|------------------|-----------------|---------|--------|
|                  | NAR             | JV      | Exact  |
| $5 \times 5$     | 0.116           | 0.00078 | 0.0045 |
| $10 \times 10$   | 0.127           | 0.0020  | 0.0053 |
| $20 \times 20$   | 0.186           | 0.0076  | 0.0094 |
| $30 \times 30$   | 0.164           | 0.014   | 0.017  |
| $50 \times 50$   | 0.259           | 0.036   | 0.051  |
| $100 \times 100$ | 0.245           | 0.135   | 0.338  |
| $200 \times 200$ | 0.260           | 0.607   | 3.69   |
| $300 \times 300$ | 0.419           | 1.44    | 11.0   |
| $400 \times 400$ | 0.783           | 2.74    | 30.9   |
| $500 \times 500$ | 1.20            | 5.36    | 52.5   |

plementation. This crossover occurs at  $(100, 100)$  for the random-distance instances and at  $(200, 200)$  for the metric and demand-weighted instances. In the separate large-scale metric experiment, NAR requires between 0.749 seconds at  $(600, 600)$  and 2.03 seconds at  $(1000, 1000)$ , while JV requires between 6.49 and 20.6 seconds on the same server instance. Exact solutions were not computed at these sizes because of their substantially higher computational cost.

These results demonstrate the runtime advantage of learned inference on sufficiently large instances. This advantage comes with a solution-quality trade-off: unlike the exact solver, NAR does not provide an optimality guarantee and generally returns solutions above the exact optimum. Nevertheless, its combination of comparatively stable solution quality and substantially lower solution time provides a favorable trade-off for large instances where exact optimization becomes computationally expensive.

Overall, the experiments show that problem size and distance structure affect the model differently. Increasing the instance size while preserving the metric structure produces only a moderate change in the NAR approximation ratio. The model is also robust to the tested client-weighting transformation and substantially outperforms its teacher under this shift. In contrast, removing the geometric structure entirely causes both methods to deteriorate, indicating that the learned procedure remains dependent on the connection-cost distribution. The timing results complement these findings: although the fixed overhead of NAR inference makes it slower on small instances, its solution time scales more favorably than both JV and exact optimization, resulting in substantial speed advantages on larger problems. NAR therefore provides a favorable solution-quality–runtime trade-off for large instances, producing comparatively stable solutions without the rapidly increasing computational cost of exact optimization.

## 6 Conclusion

This work investigated whether Neural Algorithmic Reasoning can be applied to the uncapacitated facility location problem by learning the computation of the Jain-Vazirani primal-dual approximation algorithm. A custom encoder-processor-decoder architecture was trained using intermediate JV execution traces together with supervision based on the final solution cost. The resulting model was able to produce feasible facility-location solutions on instances substantially larger than those encountered during training.

On metric instances, the NAR model generally produced solutions of slightly lower quality than JV. Nevertheless, its approximation ratio remained comparatively stable as the problem size increased. This behavior was observed up to configurations of (500, 500) when comparison with the exact optimum was available, and the model continued to exhibit a scaling trend similar to JV on instances up to (1000, 1000). These results provide evidence that the learned computation can be applied across problem sizes rather than being restricted to the small configurations used during training.

The demand-weighted experiments provide further evidence that the model learned a transferable procedure rather than simply reproducing the outputs of its teacher. Although the model was trained exclusively on metric instances generated using unweighted Euclidean distances, its solution quality remained comparatively stable after client-specific weights were introduced. Under this distribution shift, JV deteriorated because the resulting costs no longer necessarily satisfied its metric assumptions, while the NAR model substantially outperformed it. The learned algorithm therefore generalized beyond the empirical performance demonstrated by its teacher on these instances.

This generalization is not unrestricted. When the connection-cost matrix was replaced by fully random values, both NAR and JV deteriorated as the instances increased in size, and the NAR model’s advantage disappeared on the largest configurations. The model therefore retains some dependence on the structure of the training distribution, even if it can tolerate certain meaningful deviations from it. Future work could investigate whether training on a broader range of distance structures, problem sizes, or teacher algorithms can extend this robustness while preserving the observed size generalization.

## References

- Q. Cappart, D. Chételat, E. Khalil, A. Lodi, C. Morris, and P. Veličković. Combinatorial optimization and reasoning with graph neural networks, Sept. 2022.
- Y. He and E. Vitercik. Primal-Dual Neural Algorithmic Reasoning, 2025.
- K. Jain and V. Vazirani. Primal-dual approximation algorithms for metric facility location and k-median problems. In *40th Annual Symposium on Foundations of Computer Science (Cat. No.99CB37039)*, pages 2–13, Oct. 1999. doi: 10.1109/SFFCS.1999.814571.
- F. Petropoulos, G. Laporte, E. Aktas, S. A. Alumur, C. Archetti, H. Ayhan, M. Battarra, J. A. Bennell, J.-M. Bourjolly, J. E. Boylan, M. Breton, D. Canca, L. Charlin, B. Chen, C. T. Cicek, L. A. Cox, C. S. M. Currie, E. Demeulemeester, L. Ding, S. M. Disney, M. Ehr Gott, M. J. Eppler, G. Erdoğan, B. Fortz, L. A. Franco, S. Frische, S. Greco, A. J. Gregory, R. P. Härmäläinen, W. Herroelen, M. Hewitt, J. Holmström, J. N. Hooker, T. Işık, J. Johnes, B. Y. Kara, Ö. Karsu, K. Kent, C. Köhler, M. Kunc, Y.-H. Kuo, J. Lienert, A. N. Letchford, J. Leung, D. Li, H. Li, I. Ljubić, A. Lodi, S. Lozano, V. Lurkin, S. Martello, I. G. McHale, G. Midgley, J. D. W. Morecroft, A. Mutha, C. Oğuz, S. Petrovic, U. Pfersch, H. N. Psaraftis, S. Rose, L. Saarinen, S. Salhi, J.-S. Song, D. Sotiros, K. E. Stecke, A. K. Strauss, İ. Tarhan, C. Thielen, P. Toth, G. V. Berghe, C. Vasilakis, V. Vaze, D. Vigo, K. Virtanen, X. Wang, R. Weron, L. White, T. V. Woensel, M. Yearworth, E. A. Yıldırım, G. Zaccour, and X. Zhao. Operational Research: Methods and Applications. *Journal of the Operational Research Society*, 75(3):423–617, Mar. 2024. ISSN 0160-5682, 1476-9360. doi: 10.1080/01605682.2023.2253852.
- P. Veličković and C. Blundell. Neural Algorithmic Reasoning. *Patterns*, 2(7):100273, July 2021. ISSN 26663899. doi: 10.1016/j.patter.2021.100273.
- P. Veličković, A. P. Badia, D. Budden, R. Pascanu, A. Banino, M. Dashevskiy, R. Hadsell, and C. Blundell. The CLRS Algorithmic Reasoning Benchmark, June 2022.